

25 Algorithm Interview Problems

Junior ծրագրավորողի հարցազրույցի խնդիրների ժողովածու

Այս ժողովածուն պարունակում է 25 իրարից տարբեր ալգորիթմային խնդիր, որ հաճախ հարցնում են junior C++ հարցազրույցներում: Ամեն խնդիր ունի՝ պայման, գաղափար (ինչպես մտածել), ամբողջական C++ լուծում, քայլ-առ-քայլ բացատրություն, և time/space complexity: Խնդիրները խմբավորված են ըստ տեխնիկայի՝ arrays, strings, two pointers, hashing, recursion, linked list, stack, binary search, և այլն:

Խոլմբ 1 — Arrays & Hashing

Խնդիր 1 — Two Sum

Պայման. Տրված է array և target թիվ: Գտիր երկու ինդեքս, որոնց էլեմենտների գումարը հավասար է target-ին:

Գաղափար. Brute force-ը $O(n^2)$ է (բոլոր զույգերը): Ավելի լավ. hashmap-ում պահիր տեսած թվերը: Ամեն թվի համար ստուգիր՝ $target - x$ արդեն տեսել ես: Մեկ անցքով $O(n)$:

```
vector<int> twoSum(vector<int>& nums, int target) {  
    unordered_map<int,int> seen;           // արժեք -> ինդեքս  
    for (int i = 0; i < nums.size(); i++) {  
        int need = target - nums[i];  
        if (seen.count(need))  
            return {seen[need], i};        // գտանք զույգը  
        seen[nums[i]] = i;  
    }  
    return {};  
}
```

Բացատրություն. `seen` -ը պահում է ամեն թիվ ու իր ինդեքսը: `nums[i]` -ի համար պետք է `need = target - nums[i]`. Եթե `need` -ը արդեն map-ում է, զույգը գտանք: Հակառակ դեպքում ընթացիկ թիվը ավելացնում ենք map:

Time: $O(n)$ | Space: $O(n)$

Խնդիր 2 — Contains Duplicate

Պայման. Ստուգիր՝ array-ում կա՞ն կրկնվող էլեմենտ:

Գաղափար. `set` -ում պահիր տեսած էլեմենտները: Եթե նորը արդեն set-ում է -> կրկնություն կա:

```
bool containsDuplicate(vector<int>& nums) {
    unordered_set<int> seen;
    for (int x : nums) {
        if (seen.count(x)) return true;    // արդեն տեսել ենք
        seen.insert(x);
    }
    return false;
}
```

Բացատրություն. Ամեն էլեմենտ ստուգում ենք set-ում. `count` -ը `O(1)` է hash set-ի համար: Առաջին կրկնության դեպքում վերադարձնում ենք `true`:

Time: $O(n)$ | Space: $O(n)$

Խնդիր 3 — Maximum Subarray (Kadane's Algorithm)

Պայման. Գտիր հարակից (contiguous) subarray-ի ամենամեծ գումարը:

Գաղափար. Kadane's algorithm. ամեն դիրքում որոշիր՝ ընթացիկ էլեմենտից նոր սկսել, թե՞ շարունակել նախորդ subarray-ը: Պահիր running max:

```
int maxSubArray(vector<int>& nums) {
    int curr = nums[0], best = nums[0];
    for (int i = 1; i < nums.size(); i++) {
        curr = max(nums[i], curr + nums[i]);    // նոր սկսել թե՞ շարունակել
        best = max(best, curr);
    }
    return best;
}
```

Բացատրություն. `curr` -ը այն max subarray-ի գումարն է, որ ավարտվում է `i` -ում: Եթե `curr + nums[i]` ավելի փոքր է քան հենց `nums[i]`, ապա ձեռնառու է նորից սկսել: `best` -ը պահում է ընդհանուր max-ը:

Time: $O(n)$ | Space: $O(1)$

Խնդիր 4 — Move Zeroes

Պայման. Բոլոր 0-երը տեղափոխիր array-ի վերջ՝ պահպանելով մյուս էլեմենտների հերթականությունը (in-place):

Գաղափար. Two pointer. `insertPos` -ը ցույց է տալիս, թե որտեղ պետք է դնել հաջորդ ոչ-զրո էլեմենտը:

```
void moveZeroes(vector<int>& nums) {  
    int insertPos = 0;  
    for (int x : nums)  
        if (x != 0)  
            nums[insertPos++] = x;    // ոչ-զրոները առաջ բեր  
    while (insertPos < nums.size())  
        nums[insertPos++] = 0;    // մնացածը լցրու 0-ով  
}
```

Բացատրություն. Առաջին անցքում բոլոր ոչ-զրո էլեմենտները սեղմում ենք array-ի սկիզբ: Ապա մնացած տեղերը լցնում 0-ով: In-place, `O(1)` լրացուցիչ հիշողություն:

Time: $O(n)$ | Space: $O(1)$

Խնդիր 5 — Group Anagrams

Պայման. Տրված է string-երի ցուցակ: Խմբավորիր anagram-ները (նույն տառերից կազմված բառերը) միասին:

Գաղափար. Anagram-ների sorted տարբերակը նույնն է (`"eat"`, `"tea"` -> `"aet"`): Օգտագործիր sorted string-ը որպես hashmap-ի key:

```
vector<vector<string>> groupAnagrams(vector<string>& strs) {  
    unordered_map<string, vector<string>> groups;  
    for (string s : strs) {  
        string key = s;  
        sort(key.begin(), key.end());    // key = sorted տառեր  
        groups[key].push_back(s);  
    }  
    vector<vector<string>> result;  
    for (auto& [key, group] : groups)  
        result.push_back(group);  
    return result;  
}
```

Բացատրություն. Ամեն բառ sort անում ենք՝ ստանալով canonical key: Նույն key-ով բառերը դնում ենք նույն խմբում: Վերջում հավաքում ենք բոլոր խմբերը:

Time: $O(n * k \log k)$ (n բառ, k = բառի երկարություն)

Space: $O(n * k)$

Խոլմբ 2 — Two Pointers & Strings

Խնդիր 6 — Valid Palindrome

Պայման. Ստուգիր՝ string-ը palindrome է՞ (միայն տառ/թիվ հաշվի առնելով, case-insensitive):

Գաղափար. Two pointer՝ մեկը սկզբից, մյուսը վերջից: Բաց թող ոչ-alphanumeric նիշերը, համեմատիր:

```
bool isPalindrome(string s) {  
    int l = 0, r = s.size() - 1;  
    while (l < r) {  
        while (l < r && !isalnum(s[l])) l++; // բաց թող  
        while (l < r && !isalnum(s[r])) r--;  
        if (tolower(s[l]) != tolower(s[r]))  
            return false;  
        l++; r--;  
    }  
    return true;  
}
```

Բացատրություն. Երկու pointer շարժվում են դեպի կենտրոն: Ոչ-alphanumeric-ները բաց ենթ թողնում: Եթե որևէ զույգ չի համընկնում -> ոչ palindrome:

Time: $O(n)$ | Space: $O(1)$

Խնդիր 7 — Valid Parentheses

Պայման. Ստուգիր՝ `()[]{}` փակագծերը ճիշտ են դասավորված:

Գաղափար. Stack. Բացվող փակագիծը push արա, փակվողի դեպքում ստուգիր՝ stack-ի վերևը համապատասխան բացվող է:

```
bool isValid(string s) {
    stack<char> st;
    unordered_map<char, char> pairs = {'(', ')', '[', ']', '{', '}'};
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') {
            st.push(c);
        } else {
            if (st.empty() || st.top() != pairs[c])
                return false;
            st.pop();
        }
    }
    return st.empty();    // բոլորը փակվե՞լ են
}
```

Բացատրություն. Բացվող փակագծերը դնում ենք stack: Փակվողի դեպքում stack-ի վերևը պետք է լինի համապատասխան բացվողը: Վերջում stack-ը պետք է դատարկ լինի:

Time: O(n) | Space: O(n)

Խնդիր 8 — Longest Substring Without Repeating Characters

Պայման. Գտիր ամենաերկար ենթատողի (substring) երկարությունը՝ առանց կրկնվող նիշերի:

Գաղափար. Sliding window. պահիր window, որտեղ բոլոր նիշերը եզակի են: Կրկնության դեպքում սեղմիր ձախ եզրը:

```
int lengthOfLongestSubstring(string s) {
    unordered_map<char, int> lastPos;    // նիշ -> վերջին ինդեքս
    int start = 0, best = 0;
    for (int i = 0; i < s.size(); i++) {
        if (lastPos.count(s[i]) && lastPos[s[i]] >= start)
            start = lastPos[s[i]] + 1;    // window-ի ձախ եզրը շարժիր
        lastPos[s[i]] = i;
        best = max(best, i - start + 1);
    }
    return best;
}
```

Բացատրություն. `start` -ը window-ի ձախ եզրն է: Երբ հանդիպում ենք կրկնվող նիշի (որը window-ում է), ձախ եզրը շարժում ենք կրկնության հաջորդ դիրք: `best` -ը պահում է ամենամեծ window-ի երկարությունը:

Time: $O(n)$ | Space: $O(\min(n, \text{alphabet}))$

Խնդիր 9 — Reverse String (in-place)

Պայման. Շրջիր char array-ը in-place (առանց լրացուցիչ array-ի):

Գաղափար. Two pointer` փոխանակիր առաջին և վերջին, մոտենալով կենտրոնին:

```
void reverseString(vector<char>& s) {
    int l = 0, r = s.size() - 1;
    while (l < r) {
        swap(s[l], s[r]);
        l++; r--;
    }
}
```

Բացատրություն. Ձախ և աջ pointer-ները փոխանակում են էլեմենտները և շարժվում դեպի կենտրոն, մինչև հանդիպեն:

Time: $O(n)$ | Space: $O(1)$

Խնդիր 10 — First Unique Character

Պայման. Գտիր string-ի առաջին չկրկնվող նիշի ինդեքսը:

Գաղափար. Երկու անգր. առաջինում հաշվիր ամեն նիշի հանդիպումների քանակը, երկրորդում գտիր առաջին 1-count նիշը:

```
int firstUniqChar(string s) {
    unordered_map<char, int> count;
    for (char c : s) count[c]++;
    for (int i = 0; i < s.size(); i++)
        if (count[s[i]] == 1)
            return i;
    return -1;
}
```


Բացատրություն. Frequency map-ը հաշվում է ամեն նիշ: Ապա string-ով անցնում ենք հերթականությամբ և վերադարձնում առաջին նիշի ինդեքսը, որի count-ը 1 է:

Time: $O(n)$ | Space: $O(1)$ (max 26 տառ)

Խոնմբ 3 — Binary Search & Sorted Arrays

Խնդիր 11 — Binary Search

Պայման. Sorted array-ում գտիր target-ի ինդեքսը (կամ -1):

Գաղափար. Ամեն քայլում կիսիր որոնման տիրույթը. համեմատիր միջին էլեմենտը target-ի հետ:

```
int binarySearch(vector<int>& nums, int target) {  
    int l = 0, r = nums.size() - 1;  
    while (l <= r) {  
        int mid = l + (r - l) / 2;      // overflow-ից խուսափում  
        if (nums[mid] == target) return mid;  
        else if (nums[mid] < target) l = mid + 1;  
        else r = mid - 1;  
    }  
    return -1;  
}
```

Բացատրություն. `l` և `r` սահմանում են որոնման տիրույթը: `mid`-ը միջինն է $(l + (r-l)/2)$ ՝ overflow-ից խուսափելու: Կախված համեմատությունից՝ կիսում ենք տիրույթը: $O(\log n)$:

Time: $O(\log n)$ | Space: $O(1)$

Խնդիր 12 — Search in Rotated Sorted Array

Պայման. Պտտված sorted array-ում (օր. `[4,5,6,7,0,1,2]`) գտիր target-ը $O(\log n)$ -ով:

Գաղափար. Modified binary search. Կեսերից մեկը միշտ sorted է: Որոշիր որ կեսն է sorted, ստուգիր target-ը դրանում է թե ոչ:

```

int search(vector<int>& nums, int target) {
    int l = 0, r = nums.size() - 1;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (nums[mid] == target) return mid;
        if (nums[l] <= nums[mid]) {                // ձախ կեսը sorted է
            if (nums[l] <= target && target < nums[mid]) r = mid - 1;
            else l = mid + 1;
        } else {                                  // աջ կեսը sorted է
            if (nums[mid] < target && target <= nums[r]) l = mid + 1;
            else r = mid - 1;
        }
    }
    return -1;
}

```

Բացատրություն. Ամեն քայլում որոշում ենք որ կեսն է sorted (`nums[l] <= nums[mid]`): Եթե target-ը sorted կեսի տիրույթում է, փնտրում ենք այնտեղ, հակառակ դեպքում մյուս կեսում:

Time: $O(\log n)$ | Space: $O(1)$

Խնդիր 13 — Find Minimum in Rotated Sorted Array

Պայման. Պատված sorted array-ում գտիր ամենափոքր էլեմենտը $O(\log n)$ -ով:

Գաղափար. Binary search. Համեմատիր `nums[mid]`-ը `nums[r]`-ի հետ՝ որոշելու թե որ կեսում է minimum-ը (pivot կետը):

```

int findMin(vector<int>& nums) {
    int l = 0, r = nums.size() - 1;
    while (l < r) {
        int mid = l + (r - l) / 2;
        if (nums[mid] > nums[r]) l = mid + 1;    // min-ը աջ կեսում է
        else r = mid;                             // min-ը mid կամ ձախ
    }
    return nums[l];
}

```

Բացատրություն. Եթե `nums[mid] > nums[r]`, ապա rotation point-ը (min-ը) mid-ից աջ է: Հակառակ դեպքում min-ը mid-ում է կամ ձախում: Loop-ը վերջում `l == r` -> min:

Time: $O(\log n)$ | Space: $O(1)$

Խոնմբ 4 — Linked List

Խնդիր 14 — Reverse Linked List

Պայման. Շրջիր singly linked list-ը:

Գաղափար. Երեք pointer `prev`, `curr`, `next`: Ամեն քայլում `curr`-ի `next`-ը ուղղիր `prev`-ին:

```
struct ListNode { int val; ListNode* next; };

ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while (curr) {
        ListNode* next = curr->next; // հիշիր հաջորդը
        curr->next = prev;           // շրջիր ուղղությունը
        prev = curr;                 // շարժիր առաջ
        curr = next;
    }
    return prev;                    // նոր head
}
```

Բացատրություն. Ամեն node-ի `next`-ը ուղղում ենք հակառակ (դեպի `prev`): `next`-ը պահում ենք, որ չկորցնենք մնացած list-ը: Վերջում `prev`-ը նոր head-ն է:

Time: $O(n)$ | Space: $O(1)$

Խնդիր 15 — Detect Cycle in Linked List

Պայման. Ստուգիր՝ linked list-ը ունի՞ cycle (օղակ):

Գաղափար. Floyd's Tortoise and Hare. Երկու pointer, մեկը 1 քայլ, մյուսը 2 քայլ: Եթե cycle կա, նրանք կհանդիպեն:

```

bool hasCycle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast && fast->next) {
        slow = slow->next;           // 1 քայլ
        fast = fast->next->next;      // 2 քայլ
        if (slow == fast)
            return true;           // հանդիպեցին -> cycle
    }
    return false;                  // fast հասավ վերջ -> cycle չկա
}

```

Բացատրություն. `fast` -ը 2 անգամ արագ է: Եթե cycle կա, `fast` -ը «շրջան կանի» և կհասնի `slow` -ին: Եթե cycle չկա, `fast` -ը կհասնի `nullptr` -ի:

Time: $O(n)$ | Space: $O(1)$

Խնդիր 16 — Merge Two Sorted Lists

Պայման. Միավորիր երկու sorted linked list մեկ sorted list-ի:

Գաղափար. Dummy node + երկու pointer: Ամեն քայլում ընտրիր ավելի փոքր head-ը:

```

ListNode* mergeTwoLists(ListNode* a, ListNode* b) {
    ListNode dummy;
    ListNode* tail = &dummy;
    while (a && b) {
        if (a->val <= b->val) { tail->next = a; a = a->next; }
        else { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b; // մնացած մասը կցիր
    return dummy.next;
}

```

Բացատրություն. `dummy` -ն օգնում է առանց հատուկ դեպքերի կառուցել արդյունքը: Ամեն քայլում կցում ենք ավելի փոքր էլեմենտը: Մեկը վերջանալիս մյուսի մնացորդը կցում ենք ամբողջությամբ:

Time: $O(n + m)$ | Space: $O(1)$

Խոմբ 5 — Stack & Recursion

Խնդիր 17 — Min Stack

Պայման. Կառուցիր stack, որ `push`, `pop`, `top`, և `getMin` անում է $O(1)$ -ով:

Գաղափար. Երկրորդ stack, որ պահում է ամեն մակարդակի minimum-ը:

```
class MinStack {
    stack<int> st;
    stack<int> minSt;    // ընթացիկ minimum-ները
public:
    void push(int x) {
        st.push(x);
        if (minSt.empty() || x <= minSt.top())
            minSt.push(x);
    }
    void pop() {
        if (st.top() == minSt.top()) minSt.pop();
        st.pop();
    }
    int top()    { return st.top(); }
    int getMin() { return minSt.top(); }
};
```

Բացատրություն. `minSt` -ը `push` է անում միայն, երբ նոր էլեմենտը $\leq$ ընթացիկ min-ին: `pop`-ի ժամանակ, եթե հանվողը min-ն է, հանում ենք minSt-ից էլ: `getMin`-ը $O(1)$ է:

Time: $O(1)$ բոլոր գործողություններ | Space: $O(n)$

Խնդիր 18 — Fibonacci (Memoization)

Պայման. Հաշվիր n -րդ Fibonacci թիվը արդյունավետ:

Գաղափար. Naive recursion-ը $O(2^n)$ է (կրկնվող հաշվարկներ): Memoization-ով պահում ենք արդեն հաշվածները $\rightarrow O(n)$:


```
int fib(int n, vector<int>& memo) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];    // արդեն հաշված
    return memo[n] = fib(n-1, memo) + fib(n-2, memo);
}
// կանչ: vector<int> memo(n+1, -1); fib(n, memo);
```

Բացատրություն. `memo`-ն պահում է հաշված արժեքները: Ամեն `fib(n)` հաշվվում է մեկ անգամ: Առանց `memo`-ի `fib(n-1)` և `fib(n-2)`՝ կրկնում են նույն ենթահաշվարկները exponential անգամ:

Time: $O(n)$ | Space: $O(n)$

Խնդիր 19 — Climbing Stairs

Պայման. n աստիճան կա: Ամեն քայլում կարող ես բարձրանալ 1 կամ 2 աստիճան: Քանի ձևով կարող ես հասնել վերև:

Գաղափար. Սա Fibonacci է քողարկված. $ways(n) = ways(n-1) + ways(n-2)$: Bottom-up DP $O(1)$ ՝ հիշողությամբ:

```
int climbStairs(int n) {
    if (n <= 2) return n;
    int prev = 1, curr = 2;
    for (int i = 3; i <= n; i++) {
        int next = prev + curr;
        prev = curr;
        curr = next;
    }
    return curr;
}
```

Բացատրություն. n -րդ աստիճան հասնում ես կամ $(n-1)$ -ից 1 քայլով, կամ $(n-2)$ -ից 2 քայլով, ուստի ձևերի քանակը դրանց գումարն է: Պահում ենք միայն վերջին երկու արժեքը:

Time: $O(n)$ | Space: $O(1)$

Խնդիր 20 — Factorial & Recursion basics

Պայման. Հաշվիր $n!$ ռեկուրսիվ և ցույց տուր base case-ի կարևորությունը:

Գաղափար. $n! = n * (n-1)!$, base case $0! = 1$: Առանց base case-ի -> անվերջ recursion (stack overflow):

```
long long factorial(int n) {  
    if (n <= 1) return 1;           // base case (ՊԱՐՏԱԴԻՐ)  
    return (long long)n * factorial(n - 1);  
}
```

Բացատրություն. Ամեն կանչ փոքրացնում է խնդիրը ($n-1$), մինչև հասնի base case-ին: Base case-ը կանգնեցնում է recursion-ը: Առանց դրա՝ անվերջ խորացում -> stack overflow:

Time: $O(n)$ | Space: $O(n)$ (recursion stack)

Խոնմբ 6 — Bit Manipulation & Math

Խնդիր 21 — Single Number

Պայման. Array-ում ամեն թիվ կրկնվում է 2 անգամ, բացի մեկից: Գտիր այդ եզակին: ($O(1)$ հիշողություն)

Գաղափար. XOR. $a \oplus a = 0$ և $a \oplus 0 = a$: Բոլորը XOR անելով՝ զույգերը չեղարկվում են, մնում է եզակին:

```
int singleNumber(vector<int>& nums) {  
    int result = 0;  
    for (int x : nums)  
        result ^= x;    // զույգերը չեղարկվում են  
    return result;  
}
```

Բացատրություն. XOR-ը commutative է. կրկնվող զույգերը միմյանց չեղարկում են ($x \oplus x = 0$), և մնում է միայն եզակի թիվը ($x \oplus 0 = x$): Հանճարեղ $O(1)$ space լուծում:

Time: $O(n)$ | Space: $O(1)$

Խնդիր 22 — Count Set Bits

Պայման. Հաշվիր, թե քանի 1-bit կա թվի binary ներկայացման մեջ:

Գաղափար. Brian Kernighan's trick. $n \& (n-1)$ հեռացնում է ամենաաջ 1-bit-ը: Կրկնիր մինչև $n=0$:

```
int countBits(int n) {  
    int count = 0;  
    while (n) {  
        n &= (n - 1);    // հեռացրու ամենաաջ 1-bit-ը  
        count++;  
    }  
    return count;  
}
```

Բացատրություն. `n & (n-1)` միշտ մաքրում է ամենափոքր set bit-ը: Loop-ը կատարվում է ճիշտ այնքան անգամ, որքան 1-bit կա՝ ուստի արագ է քան բոլոր 32 bit-ը ստուգելը:

Time: $O(\text{number of set bits})$ | Space: $O(1)$

Խնդիր 23 — Reverse Integer

Պայման. Շրջիր integer-ի թվանշանները (օր. `123 -> 321`, `-456 -> -654`): Overflow-ի դեպքում վերադարձրու 0:

Գաղափար. Ամեն քայլում վերցրու վերջին թվանշանը (`% 10`), ավելացրու արդյունքին (`* 10 + digit`): Ստուգիր overflow-ը:

```
int reverse(int x) {
    long result = 0;
    while (x != 0) {
        int digit = x % 10;
        result = result * 10 + digit;
        x /= 10;
    }
    if (result > INT_MAX || result < INT_MIN) return 0; // overflow
    return (int)result;
}
```

Բացատրություն. `x % 10` -ը վերջին թվանշանն է (նշանի հետ), `x /= 10` -ը հեռացնում է այն: `result * 10 + digit` -ը կառուցում է շրջված թիվը: `long` -ը օգնում է overflow-ը բռնել:

Time: $O(\log x)$ (թվանշանների քանակ) | Space: $O(1)$

Խոլմբ 7 — Matrix & Miscellaneous

Խնդիր 24 — Rotate Matrix 90°

Պայման. $n \times n$ մատրիցը պտտիր 90 աստիճան ժամսլաքի ուղղությամբ (in-place):

Գաղափար. Երկու քայլ. (1) transpose (տողերն ու սյունները փոխանակիր), (2) ամեն տող շրջիր հորիզոնական:

```
void rotate(vector<vector<int>>& m) {
    int n = m.size();
    for (int i = 0; i < n; i++)                // transpose
        for (int j = i + 1; j < n; j++)
            swap(m[i][j], m[j][i]);
    for (int i = 0; i < n; i++)                // reverse ամեն տող
        reverse(m[i].begin(), m[i].end());
}
```

Բացատրություն. Transpose-ը $m[i][j]$ -ն դարձնում է $m[j][i]$ (հայելային անկյունագծով): Այս ամեն տողի reverse-ը ավարտում է 90° պտույտը: In-place, առանց լրացուցիչ մատրիցի:

Time: $O(n^2)$ | Space: $O(1)$

Խնդիր 25 — Merge Intervals

Պայման. Տրված են intervals (օր. $[[1,3],[2,6],[8,10]]$): Միավորիր համընկնող (overlapping) interval-ները:

Գաղափար. Sort ըստ start-ի: Այս անցիր հերթականությամբ. եթե ընթացիկ interval-ը համընկնում է վերջինի հետ, միավորիր, հակառակ դեպքում ավելացրու նորը:

```

vector<vector<int>>> merge(vector<vector<int>>>& intervals) {
    sort(intervals.begin(), intervals.end());
    vector<vector<int>>> result;
    for (auto& iv : intervals) {
        if (result.empty() || result.back()[1] < iv[0])
            result.push_back(iv);           // չի համընկնում
        else
            result.back()[1] = max(result.back()[1], iv[1]); // միավորիր
    }
    return result;
}

```

Բացատրություն. Sort-ից հետո overlapping interval-ները հարևան են: Եթե ընթացիկի start-ը $\leq$ վերջին merged-ի end-ին, ուրեմն համընկնում են $\rightarrow$ ընդլայնիր end-ը: Հակառակ դեպքում սկսիր նոր interval:

Time: $O(n \log n)$ (sort-ի պատճառով) | Space: $O(n)$

Ամփոփ — Տեխնիկաների աղյուսակ

#	Խնդիր	Տեխնիկա	Time	Space
1	Two Sum	Hashmap	$O(n)$	$O(n)$
2	Contains Duplicate	Hash set	$O(n)$	$O(n)$
3	Maximum Subarray	Kadane / DP	$O(n)$	$O(1)$
4	Move Zeroes	Two pointer	$O(n)$	$O(1)$
5	Group Anagrams	Hashmap + sort	$O(nk \log k)$	$O(nk)$
6	Valid Palindrome	Two pointer	$O(n)$	$O(1)$
7	Valid Parentheses	Stack	$O(n)$	$O(n)$
8	Longest Substring	Sliding window	$O(n)$	$O(n)$
9	Reverse String	Two pointer	$O(n)$	$O(1)$
10	First Unique Char	Frequency map	$O(n)$	$O(1)$
11	Binary Search	Binary search	$O(\log n)$	$O(1)$
12	Search Rotated	Binary search	$O(\log n)$	$O(1)$
13	Find Min Rotated	Binary search	$O(\log n)$	$O(1)$
14	Reverse Linked List	Pointers	$O(n)$	$O(1)$
15	Detect Cycle	Fast/slow pointer	$O(n)$	$O(1)$
16	Merge Sorted Lists	Two pointer	$O(n+m)$	$O(1)$
17	Min Stack	Auxiliary stack	$O(1)$	$O(n)$
18	Fibonacci	Memoization / DP	$O(n)$	$O(n)$
19	Climbing Stairs	DP	$O(n)$	$O(1)$
20	Factorial	Recursion	$O(n)$	$O(n)$
21	Single Number	XOR	$O(n)$	$O(1)$
22	Count Set Bits	Bit trick	$O(\text{bits})$	$O(1)$
23	Reverse Integer	Math	$O(\log x)$	$O(1)$
24	Rotate Matrix	Transpose+reverse	$O(n^2)$	$O(1)$
25	Merge Intervals	Sort + greedy	$O(n \log n)$	$O(n)$

Interview խորհուրդներ

Միշտ բարձրաձայն մտածիր — Բացատրիր քո մոտեցումը նախքան կոդ գրելը: Interviewer-ը ուզում է տեսնել մտածելակերպդ, ոչ միայն վերջնական կոդը:

Սկսիր brute force-ից — Ասա naive լուծումը, ապա բարելավիր (օր. $O(n^2)$ -> hashmap-ով $O(n)$):

Հարցրու edge case-երի մասին — Դատարկ array, մեկ էլեմենտ, negative թվեր, overflow, null pointer:

Ասա complexity-ն — Ամեն լուծման վերջում ասա time և space complexity-ն:

Ամենահաճախ pattern-ները — Two pointers, sliding window, hashmap, binary search, fast/slow pointer, DP/memoization: Այս 6-ը ծածկում են junior խնդիրների մեծ մասը: